

The background of the slide is a dark, textured blue with numerous red squares of varying sizes scattered across it. In the center, there is a glowing white square with horizontal lines below it, resembling a computer screen or a data visualization. A person wearing a dark hoodie and pants is standing on the left side, facing away from the viewer and looking towards the glowing square.

Paywall Optional:

Recursive Request Exploits



Farzan Karimi

Agenda

- The Paywall Problem
- Traditional Bypass Methods
- **Introducing RRE:**
 - Principle of First Reference
 - Recursive Request Exploitation
 - Technique & Challenges
 - Entropy
 - Exploit
 - Sports Streaming Service
- Tool Drop!



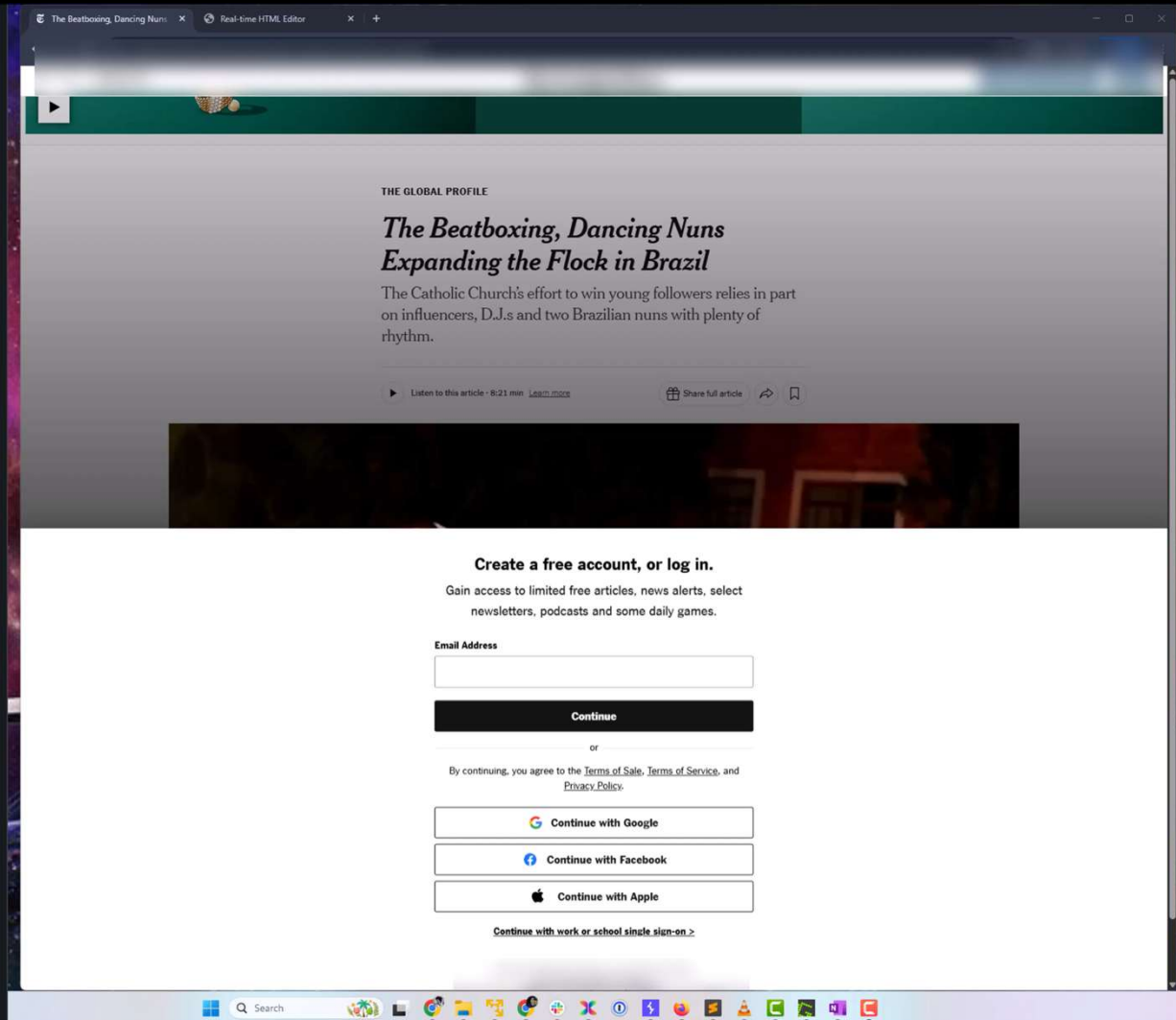
Traditional Paywall Bypass Methods

Traditional Methods

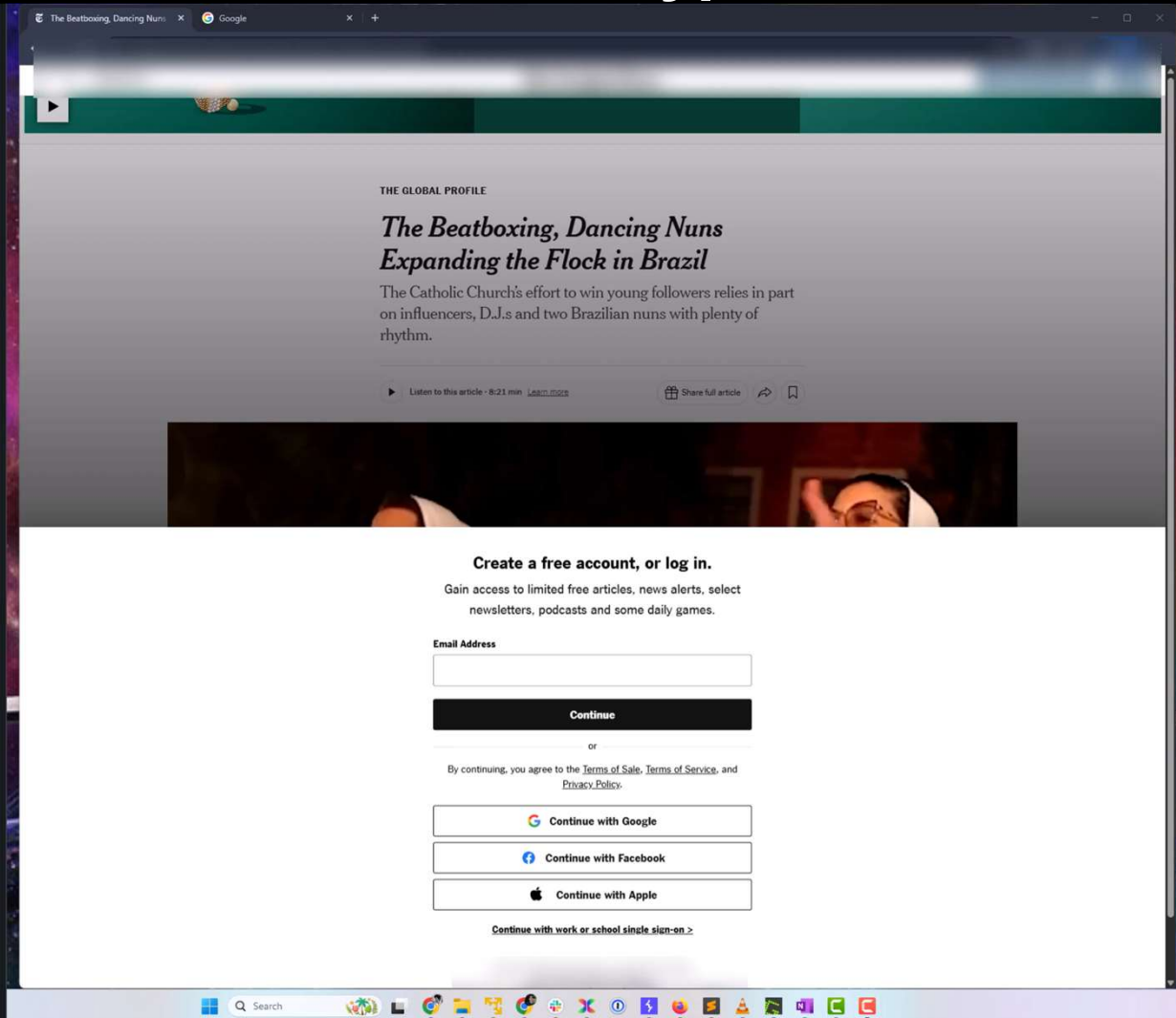
- Source Inspection
- SEO Bypass
- Referer Spoofing
- Insecure Direct Object Reference (IDOR)



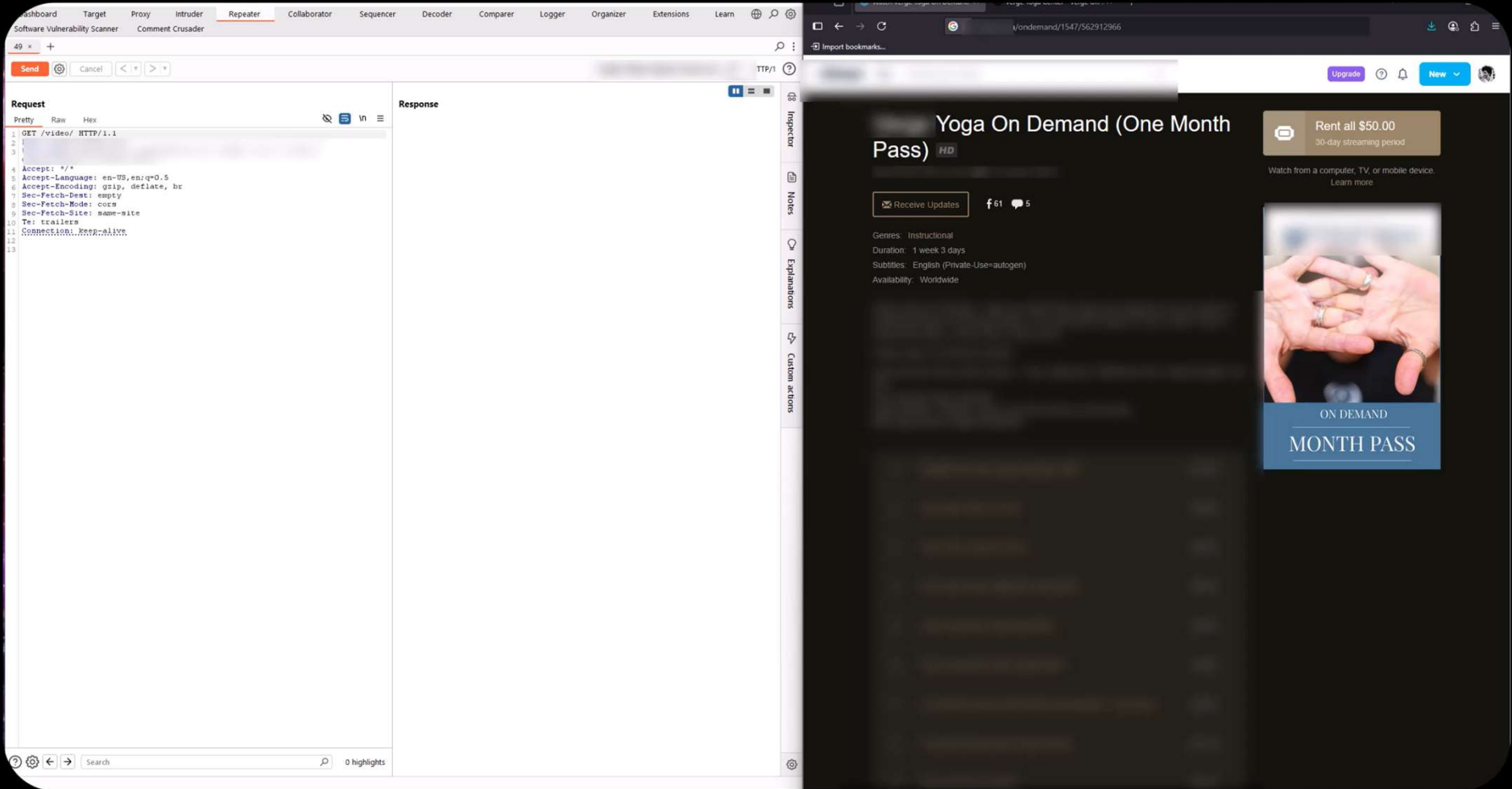
#1 Source Inspection



#2 SEO Bypass



#3 Referrer Spoofing



#4 Classic AppSec Flaws

- Insecure Direct Object Reference
- In Dir Ob Ref...
- In Dir OR...
- **IDOR**

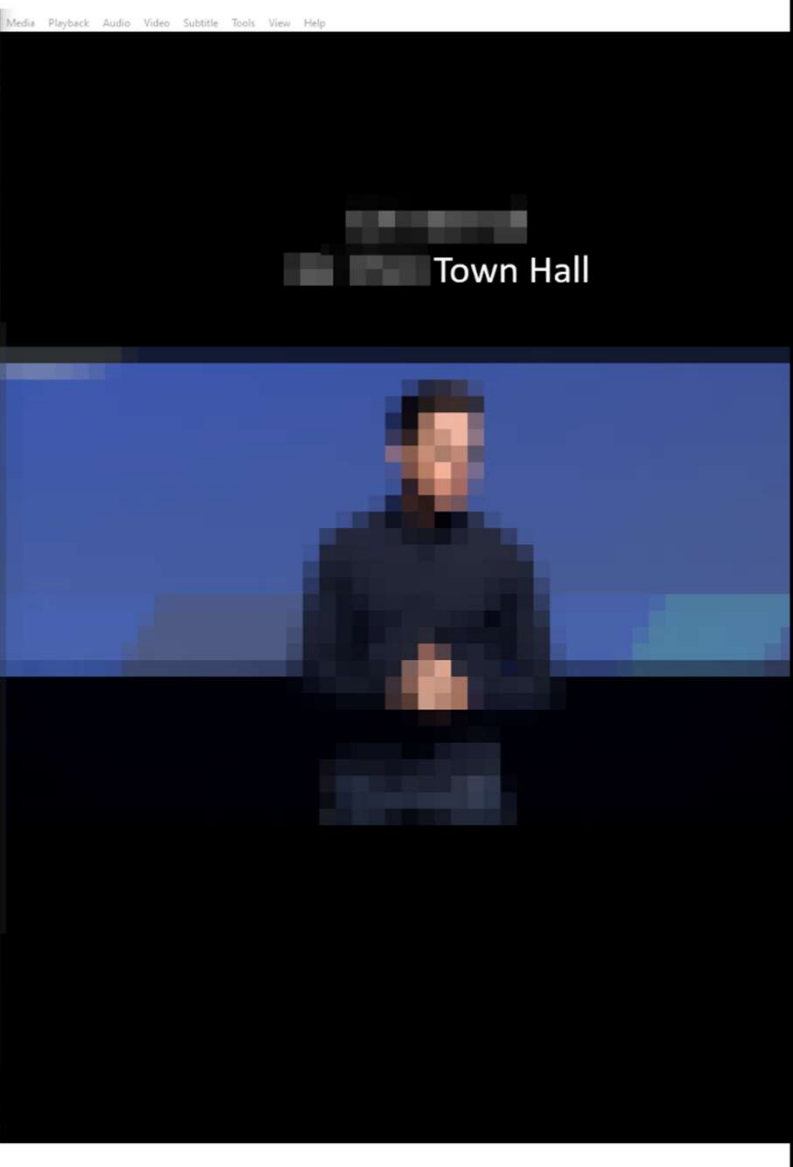


GET /accounts/10403258/events/8830666/videos/198631093/media HTTP/1.1

Request

Response

1,004 bytes | 154 millis



Send Cancel < >

Request **ORG ID**

Raw Headers Hex

GET /accounts/10403258 HTTP/1.1

Response

Raw Headers Hex

HTTP/1.1 200 OK

Internal Live Media Broadcasting

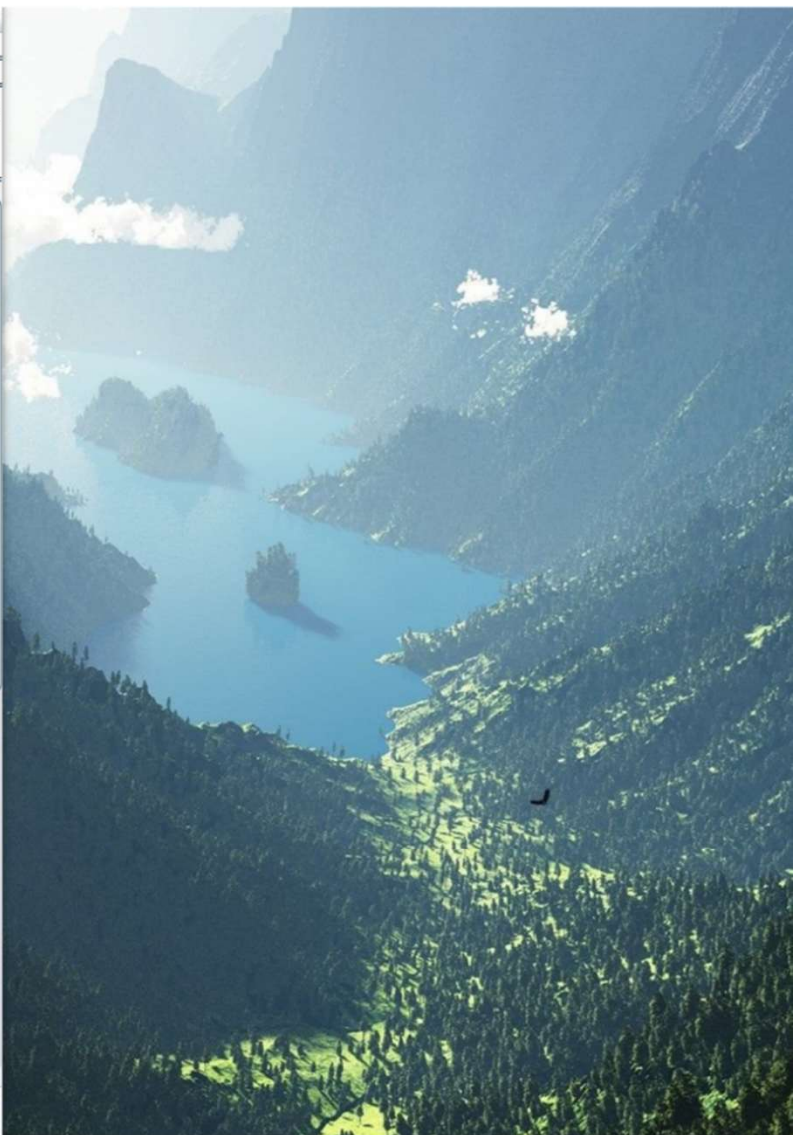
Internal Live Media Broadcasting, "background_image": {"original_width": 1280, "original_height": 960, "url": "http://accounts/10403258/events/8830666"}

accounts/10403258/events/8830666

Type a search term 0 matches

Done

9,057 bytes | 223 millis



Send Cancel < > ?

Request **ORG ID EVENT ID**

Raw Headers Hex

GET /accounts/10403258/events/8830666 HTTP/1.1

Response

Raw Headers Hex

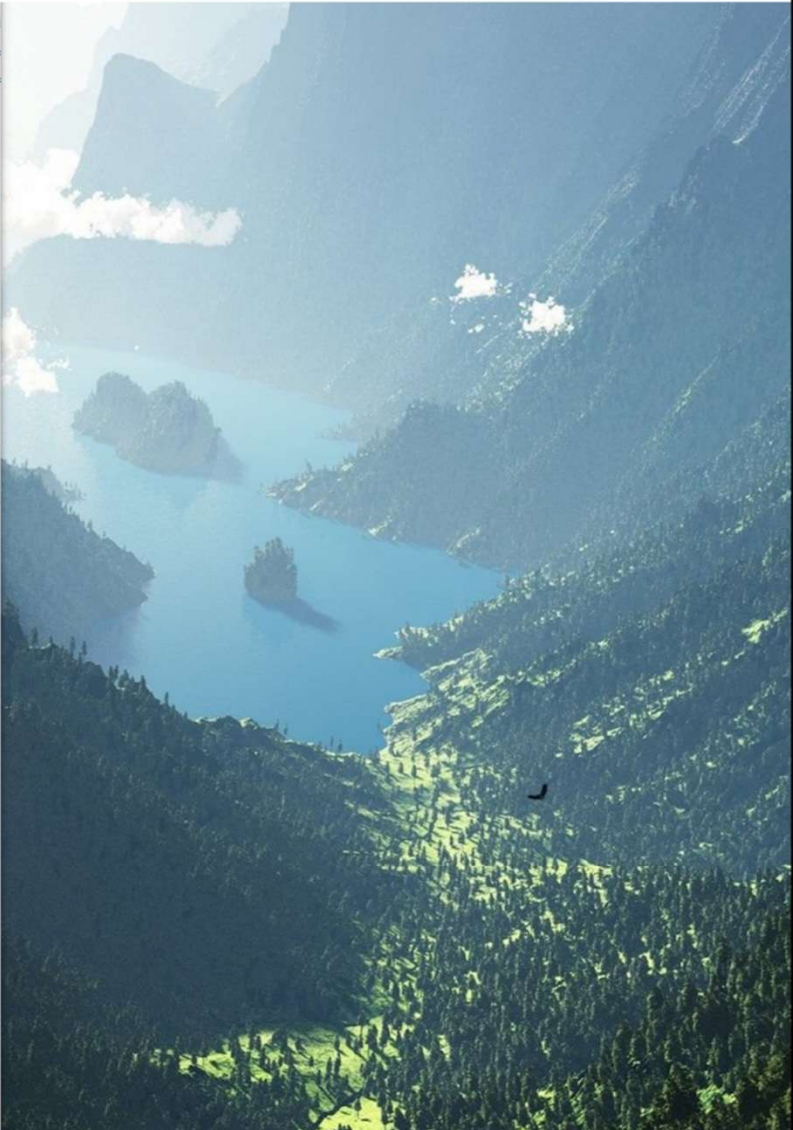
HTTP/1.1 200 OK

Town Hall

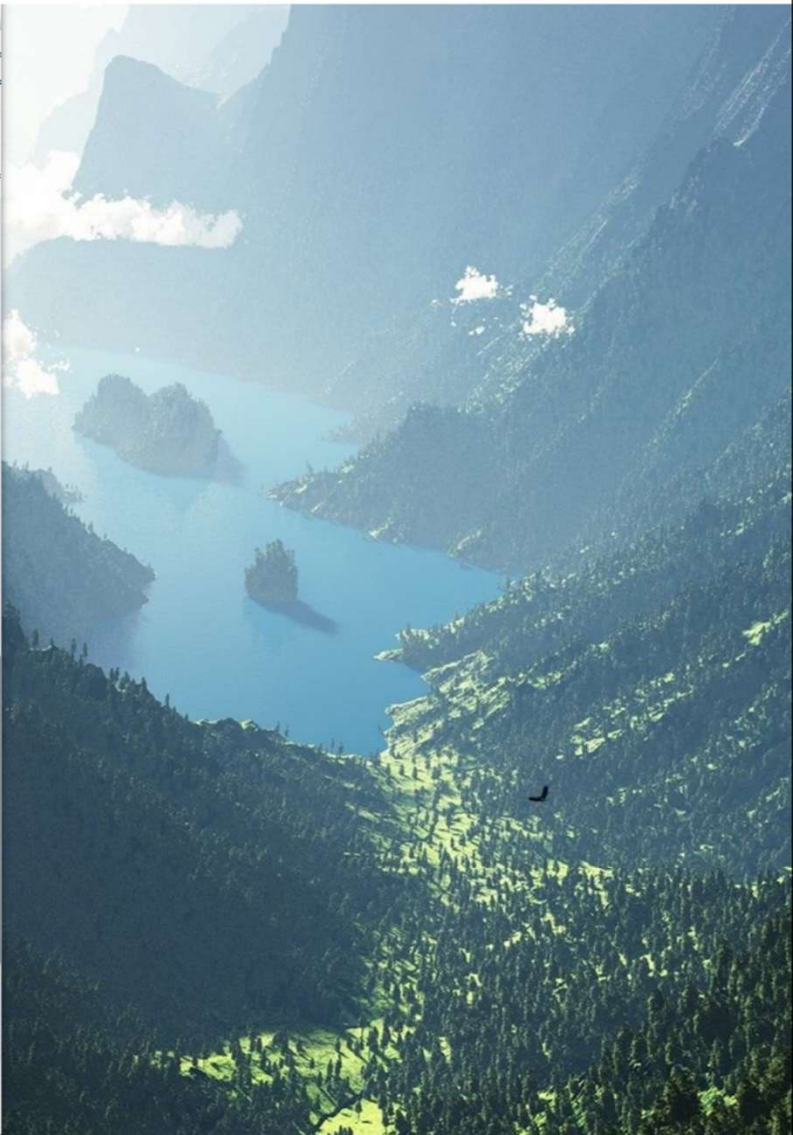
Global Town Hall

Internal Live Media Broadcasting

14,792 bytes | 300 millis



Burp Project Intruder Repeater Window Help
 Dashboard Target Proxy Intruder Repeater Sequencer Decoder Comparer Extender Project options User options
 1 2 ...
 Send Cancel < >
 Target:
 Request
 Raw Headers Hex
 GET /accounts/10403258/events/8830666 HTTP/1.1
 Response
 Raw Headers Hex
 (Scroll Down)
 GET /accounts/10403258/events/8830666
 /videos/198631093
 Direct Video Links
 0 matches
 4 matches
 14,792 bytes | 154 millis

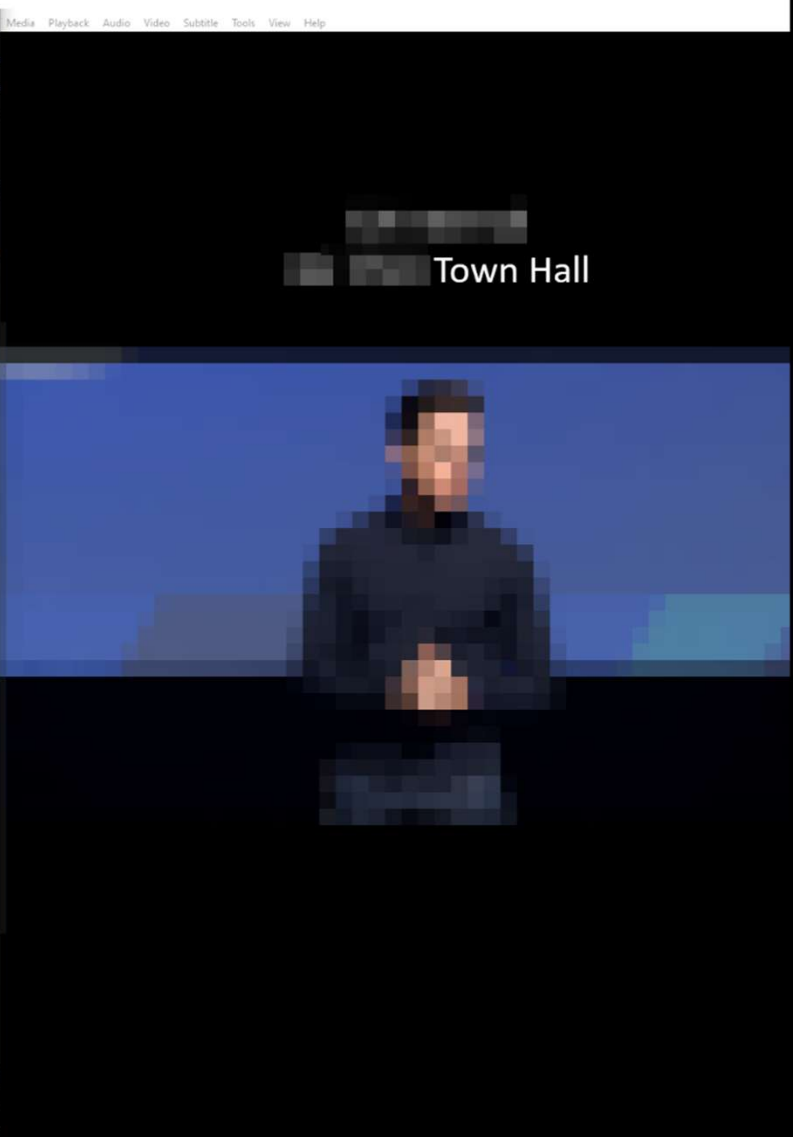


GET /accounts/10403258/events/8830666/videos/198631093/media HTTP/1.1

Request: **ORG ID** **EVENT ID** **MEDIA ID**

Response:

1,004 bytes | 154 millis



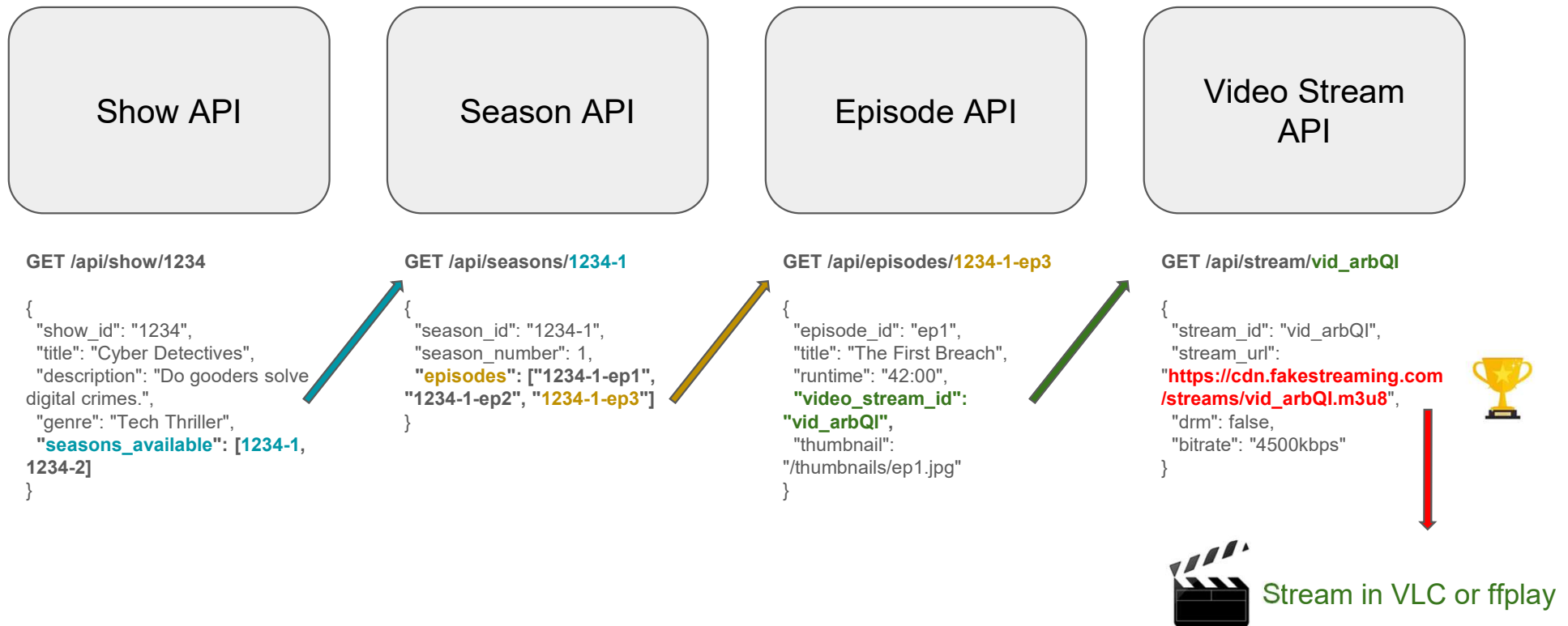


A Mental Model

AppSec Principal of First Reference

- In any user workflow, **identify the first point where a sensitive value is introduced upstream.**
- This is often where the trust assumptions are strongest and where authN/Z is weakest.
- Sensitive parameters don't randomly appear, they're generated by prior calls based on context:
 - **High-trust inputs:** e.g., authentication tokens
 - **Low-trust inputs:** e.g., metadata
- **Attacker Goal:** Trace back to the first request that introduced the sensitive parameter.
- **Challenge:** In complex apps and APIs, sensitive values are passed across nested, asynchronous calls and reused in multiple workflows - making tracing difficult.

Chaining APIs for Data Leaks



Introducing RRE!

Recursive Request Exploits:

RRE is a methodology for recursively tracing interdependent web requests in reverse to discover how sensitive values are generated.

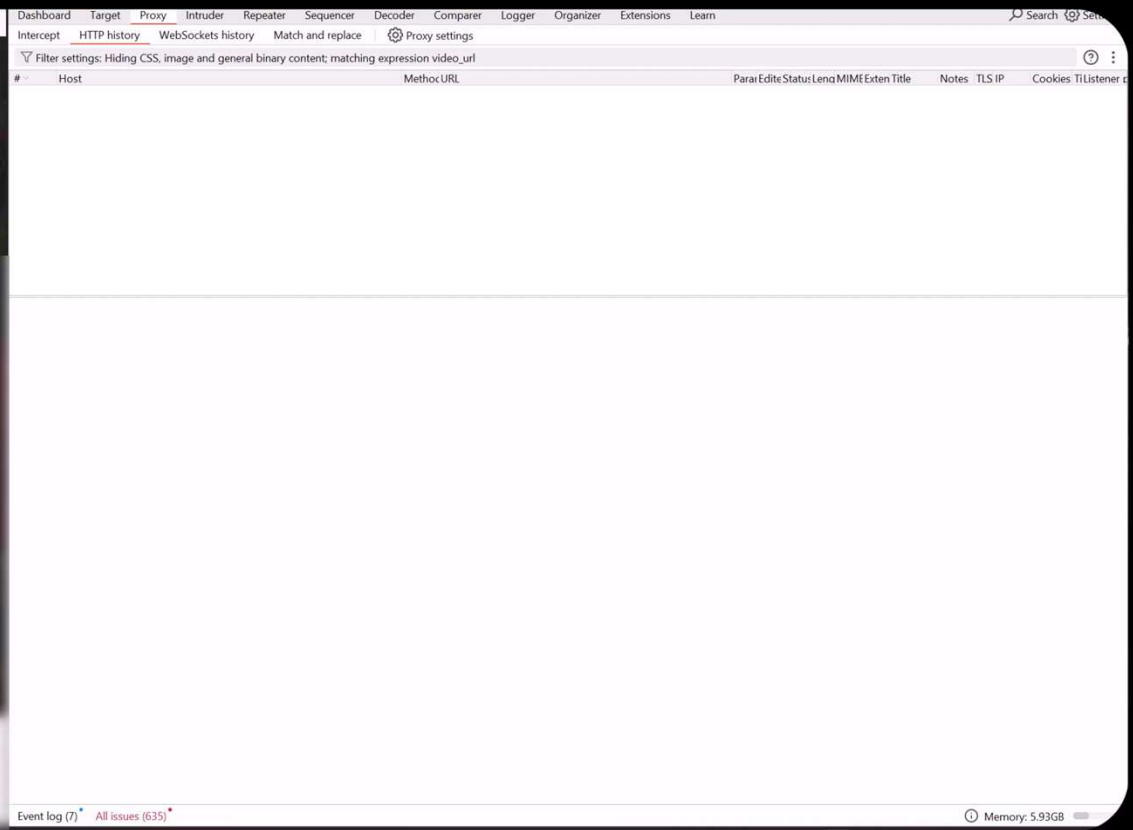
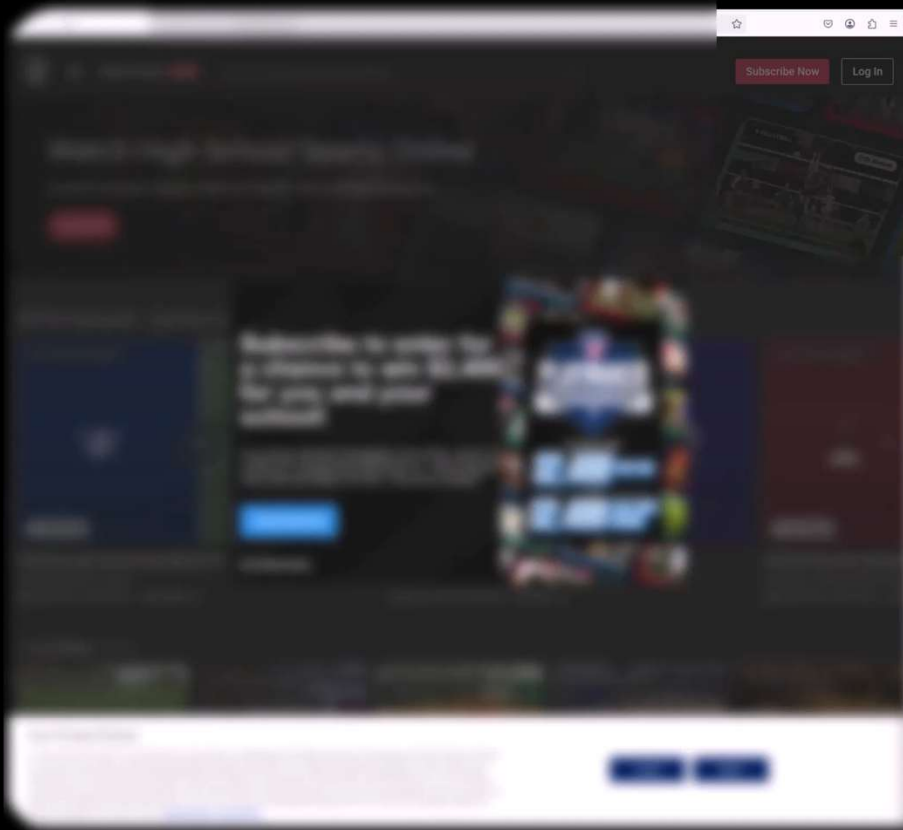
In other words: How is the first reference of a parameter of interest X generated? Now recursively ask that question.

Why? Allows attackers to systematically bypass authentication, entitlement, and payment controls. Uncovers hidden business logic flaws behind API chains.





Demo (v1): Manual Flow



Visualizing the Chain

[DEBUG] Goal Request: <https://domain.broadcast/bdc335e6be02a/vod.m3u8> → Loads Media



[DEBUG] Chained Request: GET /v2/vods/voda9b4c0aec0

[DEBUG] Highlighted Value extracted from request: voda9b4c0aec0

[DEBUG] Starting trace for value: voda9b4c0aec0



[DEBUG] Chained Request: GET /v2/search/events/?key=gam781023cb0f

[DEBUG] Highlighted Value extracted from request gam781023cb0f

[DEBUG] Starting trace for value: gam781023cb0f



[DEBUG] Chained Request: GET /v2/search/events/ondemand [No Authentication]

2 Challenges in RRE v1

Scaling Issues:

Problem 1: Couldn't automatically detect the "magic parameter"

- RRE v1 showed which APIs were part of the entitlement generation chain, **but not which parameter** triggered each sensitive value
- **Ex:** I saw **media url** came from **/v2/vods** but I had to manually identify how **voda*** and **gam*** parameters were passed

Problem 2: No recursion

- RRE v1 ran 1 layer deep
- Couldn't walk backward through the API chain in a scalable way

Solution: Shannon Entropy

Using Entropy to Spot Sensitive Values

- **Why?** Shannon entropy measures how random or unpredictable a string is, based on character frequency.

This makes it useful for spotting tokens and keys.

$$H = - \sum p(x) \log p(x)$$

- Strings with more uniform, unpredictable distributions have higher entropy (e.g. tokens):
 - `9Xh3VzXG93Ax1dK0` =  high entropy
 - `game734dElfsi` =  medium (grey) entropy
 - `admin=true` =  low entropy
- **Implemented a tunable threshold in RRE code**
 - Tuning helps reduce false positives / false negatives to catch grey cases

Entropy Threshold Calculation

```
122 def full_walkback_chain(self, value):
123     history = self._callbacks.getProxyHistory()
124     visited = set()
125
126     def calculate_shannon_entropy(s):
127         prob = [float(s.count(c)) / len(s) for c in set(s)]
128         return -sum([p * math.log(p, 2) for p in prob])
129
130     def recursive_chain(current_value):
131         if current_value in visited:
132             return
133         visited.add(current_value)
134
135         for item in history:
136             request = self._helpers.bytesToString(item.getRequest())
137             response = self._helpers.bytesToString(item.getResponse())
138
139             if not request or not response:
140                 continue
141
142             if current_value in response:
143                 top_line = request.splitlines()[0] if request else "[No Request Found]"
144                 print("***[SUCCESSFUL RRE CHAIN]***: {}".format(top_line))
145
146                 matches = re.findall(r"/([a-zA-Z0-9]{10,})", top_line)
147                 for match in matches:
148                     entropy = calculate_shannon_entropy(match)
149                     if entropy > 3.0 and match not in visited:
150                         print("[DEBUG] → High entropy match: {} (entropy: {:.2f})".format(match, entropy))
151                         recursive_chain(match)
152
153                 dependency = self.extract_dependency(response, current_value)
154                 if dependency and isinstance(dependency, str):
155                     if dependency not in visited:
156                         print("[DEBUG] → Recursing to dependency: {}".format(dependency))
157                         recursive_chain(dependency)
158                 return
159
160     print("[DEBUG] No earlier reference found for: {}".format(current_value))
```

Now time to recurse with RRE v2...

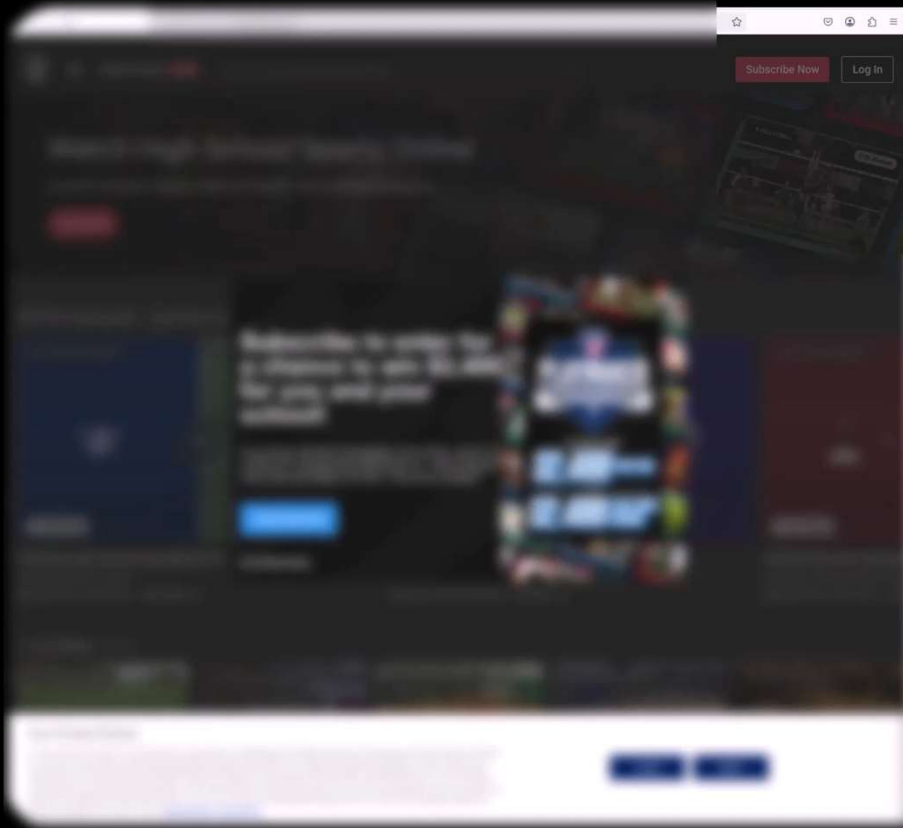
2

1

3



Demo (v2):
Recursion



Dashboard Target Proxy Intruder Repeater Sequencer Decoder Comparer Logger Organizer Extensions Learn

Intercept HTTP history WebSockets history Match and replace Proxy settings

Filter settings: Hiding CSS, image and general binary content; matching expression video_url

#	Host	Method	URL	Para	Edit	Status	Len	MIME	Ext	Title	Notes	TLS	IP	Cookies	TLS	Listener
---	------	--------	-----	------	------	--------	-----	------	-----	-------	-------	-----	----	---------	-----	----------

Event log (7) All issues (635) Memory: 5.93GB



Python

Recursive Request Exploit

Low

Details Output Errors

☐ Output to system console

☐ Save to file:

☒ Show in UI:

```
1
2 → Starting Recursive Chain Discovery
3 Initial Target: vod087de04a47    < Target Parameter
4
5 → Found in: GET /v2/game_or_event/gam60bc057ec1 HTTP/2    < Chained API
6   | High entropy match: gam60bc057ec1 (entropy: 3.39)
7   → Found in: GET /v2/highlights/hlt51f50cee30 HTTP/2    < Chained API
8   | High entropy match: hlt51f50cee30 (entropy: 3.24)
9   → Found in: GET /v3/search/highlights?card=true&size=12&cursor=initial HTTP/2    < Source API (No Auth)
10
```

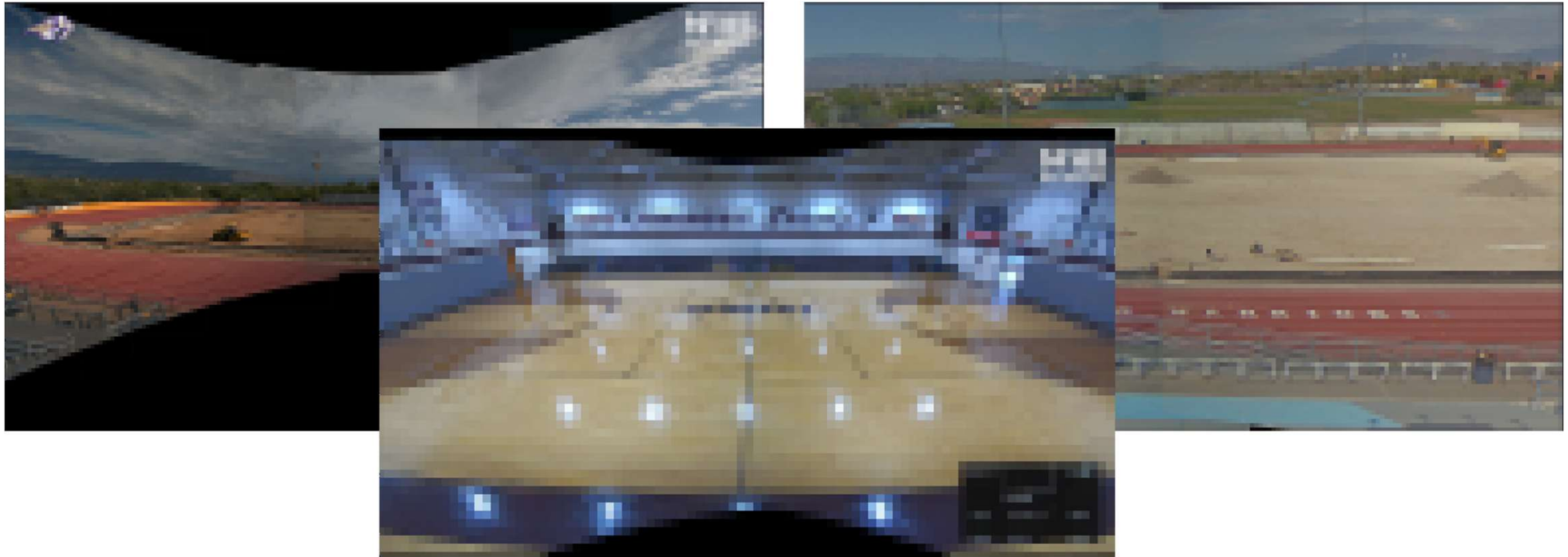
Streamageddon

- Run Exploit Code to:
1. Pull Event IDs (GamIDs)
 2. Exchange GamIDs for Broadcast IDs
 3. Exchange Broadcast IDs for direct video links



Empty Arenas?

Always-On Live Streams?



Defending Live Streams

Enforce Strong Authentication at every step

- Every API that contributes to an entitlement should enforce authN/Z (e.g., user session, auth token)

Don't Expose Sensitive Parameters via Low-Trust Inputs

- Public metadata (e.g., episode slug) should never generate access grants

Use Short-Lived Scoped Token

- Entitlement tokens should expire quickly and be bound to a session.

Enforce DRM + Session Binding for Streams

- Streaming via token alone is often bypassable
- DRM + Secure Authentication (cookies/token) + encrypted manifests raise the bar

Always On, Always Leaks

- Idle streams are still unnecessary attack surface. Gate them until showtime.

Burp Extension, Next Steps

Code

- Github: jumpycastle@
- Repo: <https://github.com/jumpycastle/rre-burp>

Next Steps

- Publish to Burp App (BApp) Store
- Miscellaneous HTTP Methods support
- Chain Discovery

Questions?



Thank you!

